

Howard IQ +

↳ Rajat Nandkumar Shedshyal

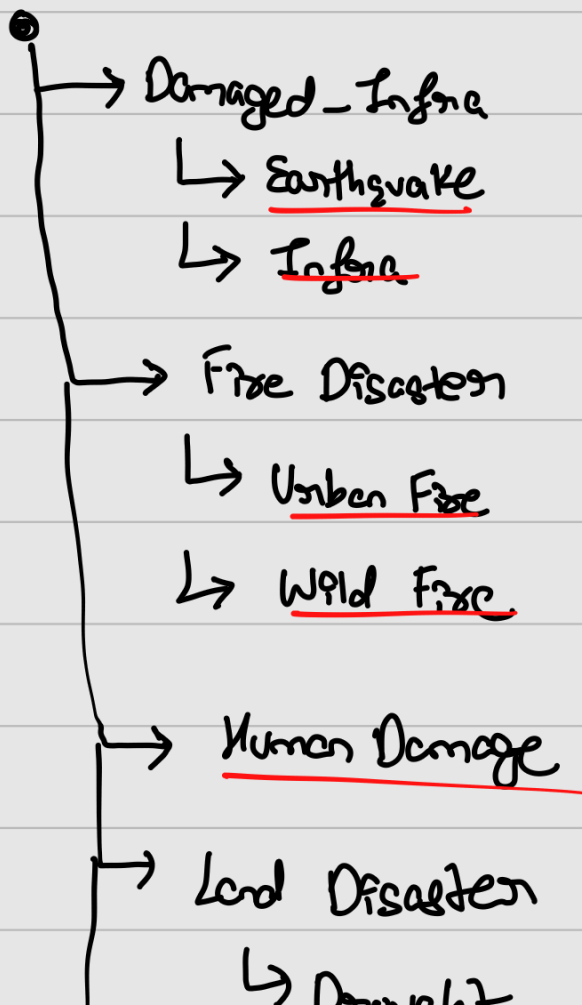
↳ after doing a lot of comparison I will be using EfficientNet - lite for fine tuning.

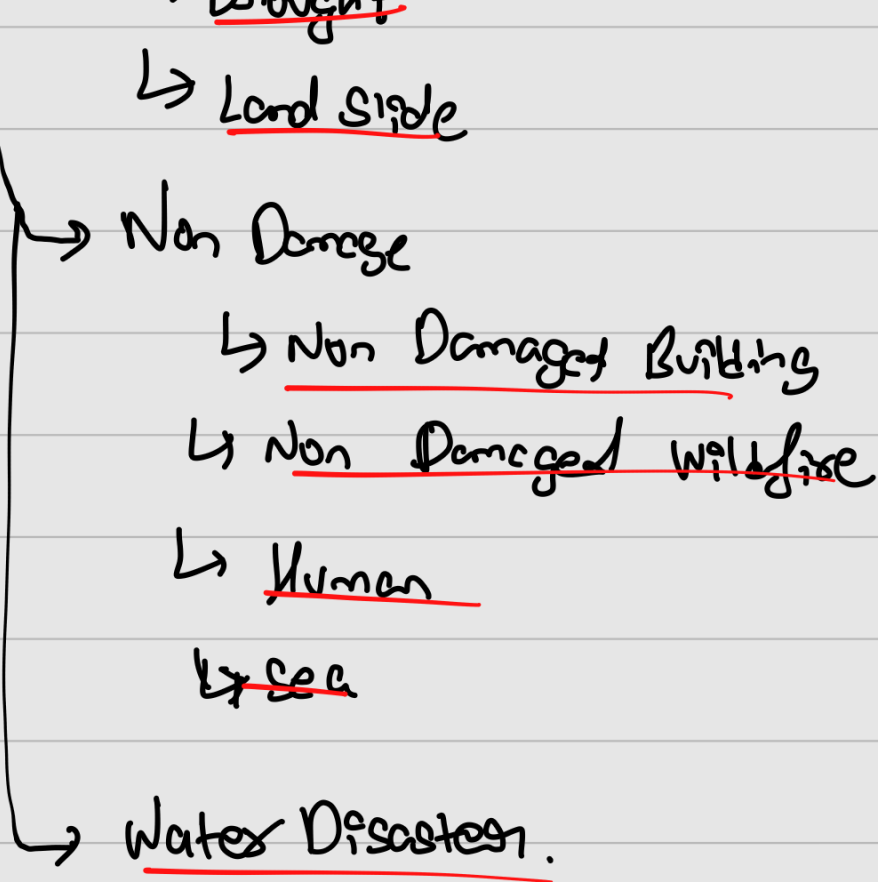
↳ Has 5 models [lite0 to lite4]

→ I will go with lite 2 for balance of Accuracy & Speed. (Best Trade off)



Dataset x Management





DataSet Prep

↳ impt parts only

`os.walk(root_dir)` → walks through every fold as sub fold

↳ returns root, subdirs, files

↓	↓	↓
Current dir	List of sub dirs	List of files

`os.path.relpath (root, root_dir)`

↳ gets folder name relative to root_dir

root_dir / bla
↳ gets this

"in" torchvision

import datasets, then we get

`datasets.folder.default_loader(path)`

sample = datasets.folder.dogcat2000.loader.ImageFolder('path')

↳ to open images

or you could use

`img = np.array (Image.open (img-path) )`

↳ But if full way
of handling
errors

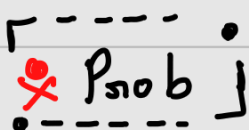
intentionally
wrong spelling 😊

$(index+1) \% len(self.samples)$ instead of $(index+1)$

↳ if $99 \% 100 = 99$

∴ Can't go
beyond last
index

↓
can go
beyond
last index



→ torch.vision doesn't have efficient resize so gotta
use "timm".

Image Pre Processing

→ resize to 224×224 [Bicubic]

→ random horizontal & vertical flip

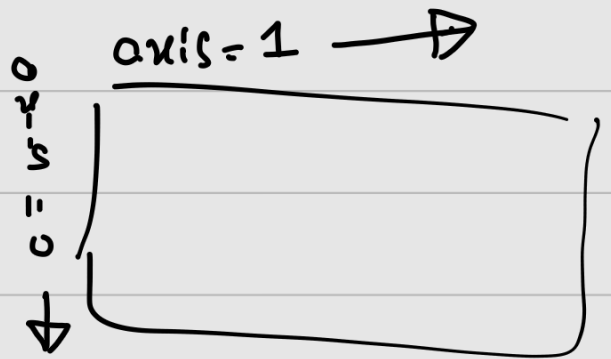
+ to tensor # makes range of pixels to $[0, 1]$
So no Normalization req like ImageNet.

Training Part

axis $\nearrow$
-, preds = torch.max(outputs, 1)

 Prob

↳ when you split into batches na sometimes the last batch may not have same size. so how to handle error calculation?



 Epoch $\rightarrow$ Send all data
Then $\rightarrow$ 1 batch sent $\rightarrow$ 1 iter complete

for batches, labels in loader:

=====

loss.item() $\rightarrow$ gives you avg loss

'''

like if its MSE & your losses individually are 10, 20, 30 then loss.item() gives 20

i.e $\frac{10+20+30}{3} = 20$ '''

So we do

$$\text{running_loss} += \text{loss.item()} * \text{inputs.size(0)}$$

...

like

$$\frac{10 + 20 + 30}{3} = 20$$

so set
sum of losses

So once your all batches are done you divide it with $\text{len}(\text{dataset})$

$$\text{actual-epoch-loss} = \text{running-loss} / \text{len}(\text{dataset})$$



PyTorch $\rightarrow$ ONNX $\rightarrow$ Tf Lite



latest tensorflow says it doesn't have
"keras. ~~src~~ engine". Problem is Onnx-tf
calls `src` from tf & current version
don't have it.

$\rightarrow$ The version mismatch is a big problem

Solution



Use virtual environment &

on the code. [Don't use Kaggle]

model → exports to ".onnx"

.onnx → .tflite using the below code.

STEP 1>

```
torch.onnx.export(model, dummy_input, "model.onnx",  
                  input_names=["input"],  
                  output_names=["output"],  
                  opset_version=11)
```

↳ gives you model.onnx

STEP 2>

download this model.onnx & then

```
main.py 2 X  model.tflite  
main.py > ...  
1 import os  
2 import onnx  
3 from onnx tf.backend import prepare  
4 import tensorflow as tf  
5  
6 # --- Step 1: Load the ONNX model ---  
7 onnx_model = onnx.load("model.onnx")  
8 print("✅ Loaded ONNX model.")  
9  
10 # --- Step 2: Convert ONNX to TensorFlow SavedModel ---  
11 saved_model_dir = "converted_saved_model"  
12 tf_rep = prepare(onnx_model)  
13 tf_rep.export_graph(saved_model_dir)  
14 print(f"✅ Converted to TensorFlow SavedModel at: {saved_model_dir}")  
15  
16 # --- Step 3: Convert SavedModel to TFLite ---  
17 converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)  
18 tflite_model = converter.convert()  
19 print("✅ Converted to TFLite format.")  
20  
21 # --- Step 4: Save the .tflite model ---  
22 tflite_path = "model.tflite"  
23 with open(tflite_path, "wb") as f:  
24 | | f.write(tflite_model)  
25 print(f"✅ Saved TFLite model to: {tflite_path}")  
26  
27 # --- Step 5: Print Input/Output Details ---  
28 interpreter = tf.lite.Interpreter(model_path=tflite_path)  
29 interpreter.allocate_tensors()  
30  
31 input_details = interpreter.get_input_details()  
32 output_details = interpreter.get_output_details()  
33  
34 print("\n🔍 Model Input(s):")  
35 for idx, i in enumerate(input_details):  
36 | | print(f" [{idx}] Name: {i['name']], Shape: {i['shape']], Dtype: {i['dtype']}")  
37  
38 print("\n🔍 Model Output(s):")  
39 for idx, o in enumerate(output_details):  
40 | | print(f" [{idx}] Name: {o['name']], Shape: {o['shape']], Dtype: {o['dtype']}")  
41  
42 print("\n✅ All Done.")  
43
```

install all dependencies first

Step 3)
[OPTIONAL]

→ you have to install "life-support" first

num of classes $\rightarrow$

for each epoch $\downarrow$

recalls =



→ change the forward pass code in class as

```
def forward (self, x):
```

is disaster = Transform (—)

yes &

Simple code

}

no &

[1,1,1,1,1,1,1,1,1,1]

}

→ EDA

↳ I don't have to do Manual Normalization as tfnn from efficient net gets the job done on its own

↳ but I don't call it as I have to normalize the images

→ We see for 5 random images that pixel intensities are not uniform so let's do something & redistribute it

Use histogram equalization

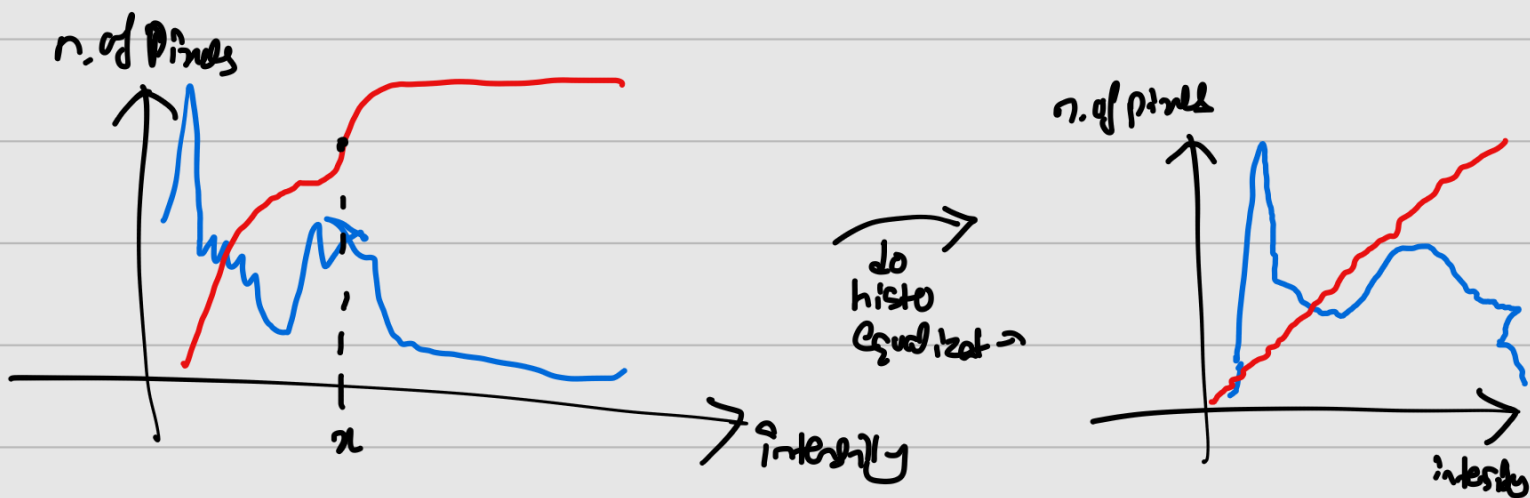
↳ a method to redistribute pixel intensities so that all intensities are equally distributed

CDF $\rightarrow$ Cumulative Distribution Function

↳ Cumulative sum of intensity values

Histo $\rightarrow$ counts for each intensity how many pixels

CDF $\rightarrow$ tells as you go on increasing how many pixels have the same intensity or less



meaning most of the pixels have intensity $< x$
so the image is darker.

CLAKE $\rightarrow$ contrast limiting adaptive equalization

$\hookrightarrow$ to enhance local contrast

1. Image is divided into small tiles / blocks
2. a histo for each tile
3. if any histo bin is above clip limit then those pixels are clipped
4. The clipped excess is redistributed uniformly to other bins before hist normalization
5. Hist normalization to each tile
6. Final Image is obtained by interpolating the tiles.

$$P_{\text{normalized}} = \frac{P - P_{\text{min}}}{P_{\text{max}} - P_{\text{min}}} \times (\text{new}_{\text{max}} - \text{new}_{\text{min}}) + \text{new}_{\text{min}}$$

Solving Imbalanced data

problems → Biasness
Metrics like accuracy are not reliable

Even a dumb model will predict good accuracy

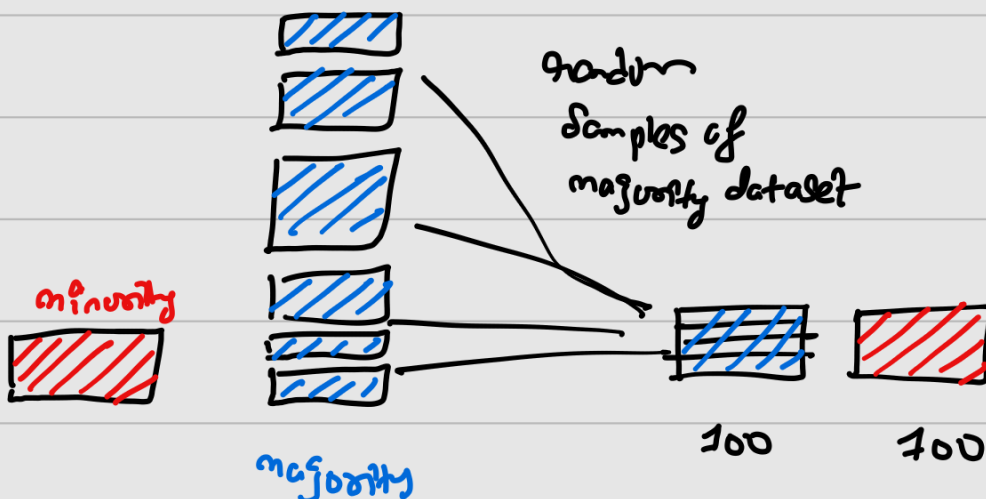
Why IMPT?

Health Care → more diseases

Finance → fraud, risk assessment

Consumer → churn prediction

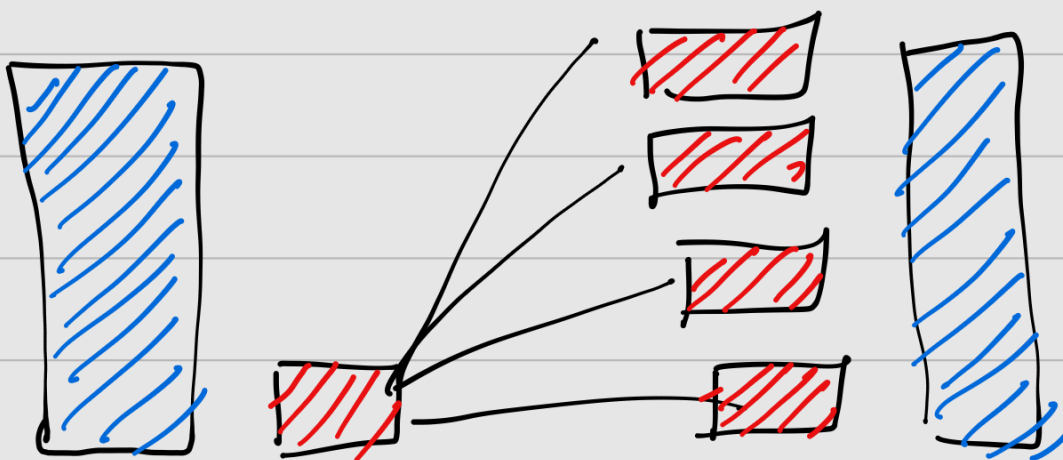
1. Undersampling



Original
Dataset

Advantages	Disadvantages
1. Reduces bias 2. Fast training	1. Info loss to underfitting 2. Sampling Bias

2. Over sampling

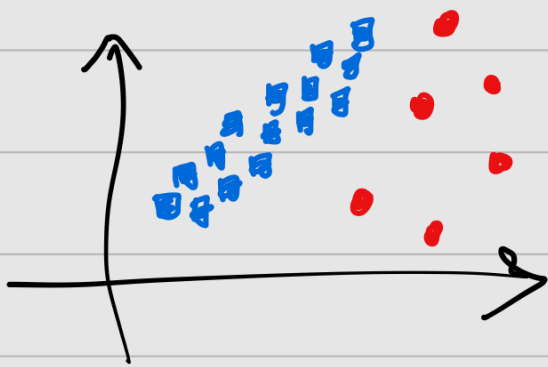


→ Copies but
random
Sampling as
before

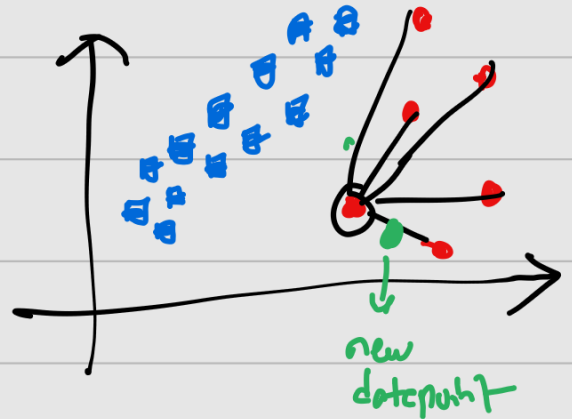
Advantages	Disadvantages
1. Reduces Bias	1. Increased size
	2. Dupl ^y of data, ^{so it} may overfit

3. SMOTE

↳ Synthetic minority oversampling technique



==>



Train a KNN on minority class obs - find 5 nearest neighbours for each

- select one at random (for the interpolation)
- Extract a random number btw 0 & 1 (factor)
- calculate the new example as

$$= \text{Org Sample} - \text{factor} * (\text{Original Sample} - \text{neighbour})$$

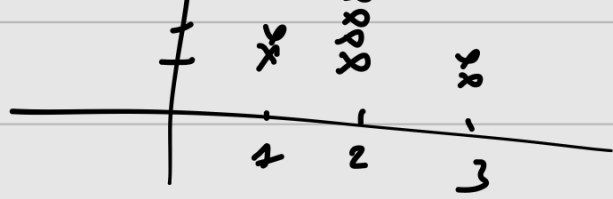
- repeat until you get minority class as equal as majority class

Advantages

Pros ↓

Disadvantages

↳ Doesn't handle categorical data



2) Computational Complexity

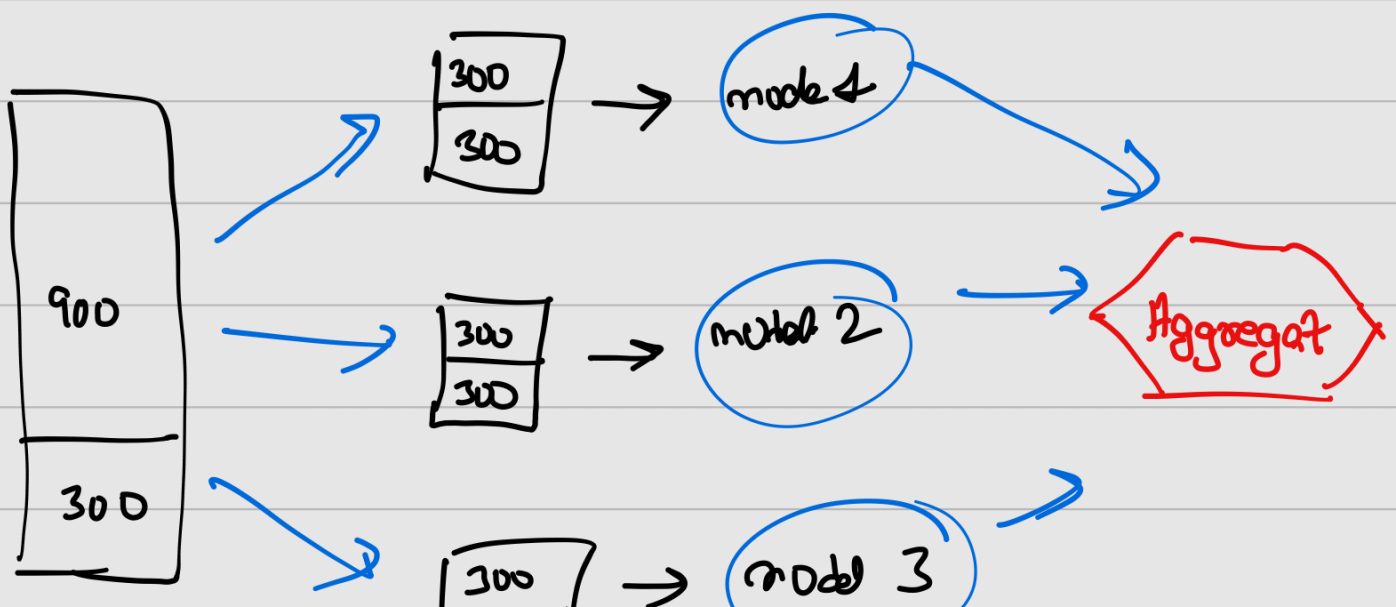
3) depends on k

4) Sensitive to outliers



5) Balance achieved may not reflect true nature.

4. Ensemble Methods



these
models are
Trees

5. Cost Sensitive

learning process $\rightarrow$ imbalanced data

1) Class weights

$[1, 0]$



weight $[1, 10]$

$\hookrightarrow$ in model only
you get this

2) Custom
loss
function

MSE
log loss etc

Ex:-

Fit
IT

+

Stratified K-fold

Class Imbalance is solved

Custom model $\hookleftarrow$

Vision Transformers 17K 2021

$\downarrow$
efficient net

}

I have added one extra class
which has no train data
just so that ViT such as Dora
refuses

[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

$\uparrow$
Fake
Disaster
Class

$\downarrow$
Test later

→ We will use nn. Cross Entropy loss
with weights

$$W_c = \frac{N}{C \cdot n_c}$$

Annotations:
- N : Total samples
- C : n. of classes
- n_c : n. of samples for c class
- W_c : weight for c class

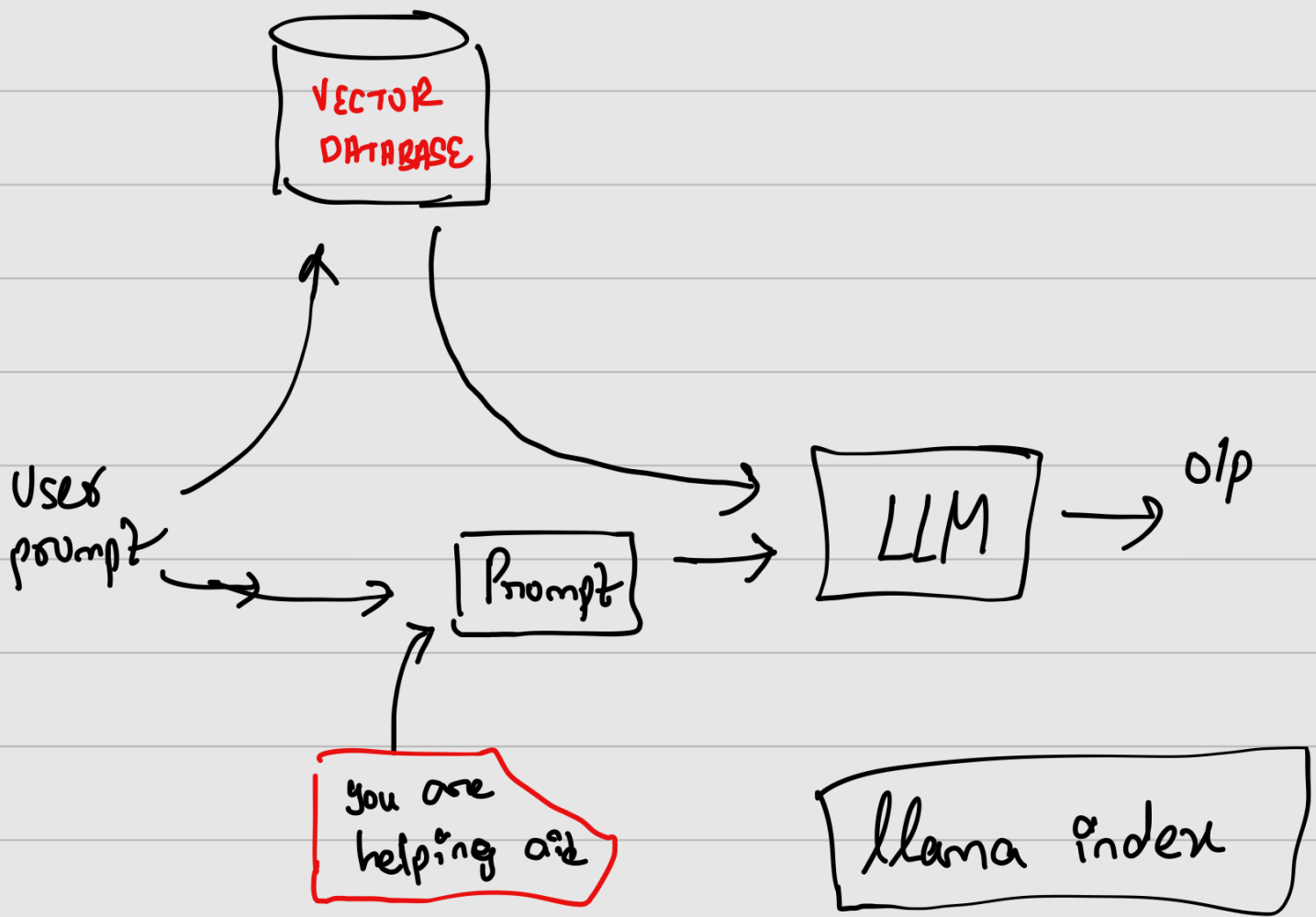
∞ ∞ nn. Cross Entropy loss

$$L = - \sum_{c=1}^C W_c \cdot y_c \log(p_c)$$

(Cross Entropy loss + Class weights)

→ Usually you can suppose to use diff

models for stratified k-Fold, but we
just wanna solve



→ use API

↳ hosted on HETEROUS

5 fold

1 epoch

↳ 2 batch

$\sum x$ n. of x $\times$ 2 batches

=

10

total $\Rightarrow$

1 batch $\Rightarrow$

213 seeds

?

? $\rightarrow$

21.3 Score

$\frac{10 \rightarrow 213}{5 \times 5 \times 10}$?

? $\rightarrow$ $213 \times 5 \times 12$ $\sum x \leq x \times 10$